

DCS Web Dashboard (Rust) — Installation & Admin Guide

A complete, start-to-finish guide for deploying the **single-binary Rust web dashboard** on a brand-new Windows server, and for administering it day to day.

This is the Rust port of the original Next.js dashboard. Unlike the Node version, it ships as **one self-contained executable** with the web UI embedded inside it — there is **no Node.js**, **no `npm`**, and **no `.env.local`** at runtime. You configure it with ordinary environment variables.

Audience: server admins setting this up for the first time. No Rust or web-dev experience required to *run* it. Building from source needs the Rust toolchain (covered below).

Table of Contents

- DCS Web Dashboard (Rust) — Installation & Admin Guide
 - Table of Contents
 - 1. What you get
 - 2. Requirements
 - 3. Get the binary
 - Option A — Use a prebuilt binary
 - Option B — Build from source
 - 4. Configuration (environment variables)
 - How to set the variables
 - 5. Discord OAuth setup
 - 6. Quick start (foreground test)
 - 7. Install as a Windows service (NSSM)
 - 8. Firewall & HTTPS
 - 9. First login
 - 10. Admin guide — features
 - 🗝️ Authentication model
 - 🟢 Server Status
 - 📖 Mission Management
 - 🌤️ Weather (*requires* `DCS_DYNAMIC_WEATHER_DIR`)
 - 📡 Radar & Atmosphere (live gRPC streams)
 - 📅 Events & 💬 Chat
 - 👤 Players
 - 💻 Tasks (Windows Task Scheduler remote control)

-  Server Settings
-  Access Logs
- 11. Updating the dashboard
- 12. Troubleshooting

1. What you get

- A single executable: `rust-web-dashboard.exe`.
- The full web UI (the Next.js static export) is **embedded** in the binary and served over HTTP, including deep links (e.g. `/weather`) and all hashed `/_next/...` assets.
- An HTTP API that is a **gRPC client** of your running DCS-gRPC server, plus filesystem/OS integration for missions, settings, weather, logs, and tasks.
- JWT bearer authentication (password and/or Discord), with a **7-day** session lifetime.

2. Requirements

On the server that runs the dashboard:

Component	Required?	Notes
Windows Server / Windows 10+	Yes	The OS-integration features (Task Scheduler, <code>serverSettings.lua</code> , missions) target Windows.
DCS-gRPC mod	Yes (for live data)	Installed in <code>Saved Games\...\Scripts</code> and running. The dashboard connects to it (default <code>http://localhost:50051</code>). Without it, the UI loads but live tabs show no data and you'll see harmless reconnect warnings in the log.
NSSM (Non-Sucking Service Manager)	Recommended	To run the binary as a resilient Windows service. Download from nssm.cc .
DCS-Dynamic-Weather + Python	Optional	Only needed for the Weather tab. Python must be reachable (on PATH or via <code>PYTHON_EXE</code>).
Node.js + Rust toolchain	Only to build	Not needed if you have a prebuilt <code>rust-web-dashboard.exe</code> . See §3.

To build the binary yourself you additionally need:

- **Rust 1.85+** (rustup.rs). No system `protoc` or Lua needed — protobuf is compiled with a bundled `protoc` , and Lua is vendored.
- **Node.js 18+** *only* if you need to rebuild the embedded frontend from `web-dashboard` .

3. Get the binary

You can either use a prebuilt binary or build from source.

Option A — Use a prebuilt binary

Copy `rust-web-dashboard.exe` to a permanent folder on the server, e.g.:

```
C:\DCS-Dashboard\rust-web-dashboard.exe
```

That single file contains the whole web UI. Nothing else needs to be deployed alongside it.

Option B — Build from source

The frontend snapshot is already embedded in the crate's `static/` folder, so a normal release build is all you need.

cwd caveat (Windows): this crate is **not** part of a cargo workspace, and terminals often open at the repo root. Always pass `--manifest-path` so cargo finds the right manifest.

```
cargo build --release --manifest-path  
"C:\Users\thierry\Documents\GitHub\DCS-gRPC\rust-web-dashboard\Cargo.toml"
```

The optimized binary is produced at:

```
rust-web-dashboard\target\release\rust-web-dashboard.exe
```

(To refresh the embedded UI after frontend changes, see §11.)

4. Configuration (environment variables)

The dashboard reads **environment variables** at startup. There is **no** `.env.local` file — empty/whitespace values are treated as unset.

The only **required** variable is `JWT_SECRET` . The binary refuses to start without a secret of at least 16 characters.

Variable	Required	Default	Purpose
<code>JWT_SECRET</code>	Yes	—	Signing secret for login tokens (HS256). Must be ≥ 16 characters. Use a long random string and keep it private. Changing it logs everyone out.
<code>DASHBOARD_ADDR</code>	No	<code>0.0.0.0:3001</code>	Address/port the dashboard listens on. Use <code>127.0.0.1:3001</code> if a reverse proxy is in front.
<code>GRPC_ENDPOINT</code>	No	<code>http://localhost:50051</code>	Where your DCS-gRPC server is listening.
<code>APP_URL</code>	No	<code>http://localhost:3001</code>	Public base URL of the dashboard. Must match the real public IP/host for Discord login redirects to work.
<code>ADMIN_PASSWORD</code>	No	—	Enables master-password login when set.
<code>MOBILE_API_KEY</code>	No	—	Static bearer key accepted by the native mobile app. Generate a random string.
<code>DISCORD_CLIENT_ID</code>	No	—	Discord OAuth — enables Discord login when all four Discord vars are set.
<code>DISCORD_CLIENT_SECRET</code>	No	—	Discord OAuth app secret.

Variable	Required	Default	Purpose
<code>DISCORD_GUILD_ID</code>	No	—	Your Discord server (guild) ID. Users must be a member.
<code>DISCORD_ADMIN_ROLE_ID</code>	No	—	Comma-separated list of role IDs allowed to log in.
<code>DCS_SAVED_GAMES_DIR</code>	No	derived	Absolute path to your DCS <i>Saved Games</i> folder. Drives <code>serverSettings.lua</code> , <code>Missions/</code> , and uploads. Set this explicitly unless the binary lives under <code>Saved Games\...\Scripts</code> .
<code>DCS_TASK_WHITELIST</code>	No	—	Comma-separated, case-insensitive allow-list of Windows scheduled-task names the Tasks tab may show/control. If unset, all root tasks are shown.
<code>DCS_DYNAMIC_WEATHER_DIR</code>	No	—	Path to the DCS-Dynamic-Weather generator folder. Required for the Weather tab.
<code>PYTHON_EXE</code>	No	<code>python</code>	Python interpreter used by the weather generator.
<code>AUDIT_LOG_PATH</code>	No	<code>audit_logs.json</code>	File where login attempts are recorded (Access Logs tab).

Variable	Required	Default	Purpose
<code>RUST_LOG</code>	No	<code>info,rust_web_dashboard=debug</code>	Log verbosity (e.g. <code>info</code> , <code>warn</code> , <code>debug</code>).

How to set the variables

Because there is no `.env` file, set them in the **process environment**. Two common ways:

(a) A launch script (good for testing; PowerShell `.ps1`):

```
# start-dashboard.ps1
$env:JWT_SECRET = "change-me-to-a-long-random-secret" # >= 16
chars
$env:APP_URL = "http://YOUR_PUBLIC_IP:3001"
$env:DASHBOARD_ADDR = "0.0.0.0:3001"
$env:GRPC_ENDPOINT = "http://localhost:50051"
$env:ADMIN_PASSWORD = "your_master_password"
$env:DCS_SAVED_GAMES_DIR = "C:\Users\admin\Saved Games\DCS.openbeta_server"
$env:DCS_TASK_WHITELIST = "DCS Web Server Console, DCSAdminBot, gRPC-LSO"
# Discord (optional):
$env:DISCORD_CLIENT_ID = "..."
$env:DISCORD_CLIENT_SECRET = "..."
$env:DISCORD_GUILD_ID = "..."
$env:DISCORD_ADMIN_ROLE_ID = "role_id_1,role_id_2"
# Weather (optional):
$env:DCS_DYNAMIC_WEATHER_DIR = "C:\Users\admin\Saved
Games\DCS.openbeta_server\Missions\Dynamic_Weather_mission"

& "C:\DCS-Dashboard\rust-web-dashboard.exe"
```

(b) The NSSM "Environment" tab (recommended for production) — see §7. This keeps secrets out of scripts and survives reboots.

⚠ Never commit `JWT_SECRET` , `ADMIN_PASSWORD` , the Discord client secret, or `MOBILE_API_KEY` to source control or paste them in chat. Treat them like passwords.

5. Discord OAuth setup

Skip this section if you only use the master password.

1. Go to the Discord Developer Portal and create (or select) an **Application**.
2. Open the **OAuth2** tab.

3. Under **Redirects**, add your callback URL — it must match **APP_URL** :

```
http://YOUR_PUBLIC_IP:3001/api/auth/callback
```

(Use `https://your.domain/api/auth/callback` if you run behind an HTTPS reverse proxy.)

4. Copy the **Client ID** and **Client Secret** into **DISCORD_CLIENT_ID** / **DISCORD_CLIENT_SECRET** .
5. Set **DISCORD_GUILD_ID** to your server ID and **DISCORD_ADMIN_ROLE_ID** to the role(s) allowed in (comma-separated for multiple).
6. Make sure **APP_URL** exactly matches the public address users hit — otherwise Discord will bounce logins back to **localhost** .

A successful login mints a **7-day** session token. If a user loses their role, they are rejected the next time their session expires or they log in from a new device.

6. Quick start (foreground test)

Before installing a service, confirm it boots and serves the UI.

```
$env:JWT_SECRET      = "change-me-to-a-long-random-secret"  
$env:DASHBOARD_ADDR = "127.0.0.1:3099"  
& "C:\DCS-Dashboard\rust-web-dashboard.exe"
```

Then open `http://127.0.0.1:3099/` in a browser. You should see the dashboard UI, and deep links like `/weather` should load directly.

- If DCS-gRPC isn't running yet, you'll see repeating warnings like *"failed to open StreamUnits/StreamEvents; retrying"* — **this is expected** and harmless; live tabs simply have no data until the gRPC server is up.
- Stop with `Ctrl-C` (graceful shutdown).

7. Install as a Windows service (NSSM)

Running the binary as a service makes it start on boot and restart on failure. NSSM works the same as it did for the Node version, except the **Path** points directly at the `.exe` and there is no `npm` /working-directory fragility.

1. Open an **Administrator** PowerShell/Command Prompt and go to your extracted NSSM folder:

```
cd C:\nssm\win64  
nssm install "DCS Web Dashboard"
```

2. In the GUI:

Application tab

- Path: `C:\DCS-Dashboard\rust-web-dashboard.exe`
- Startup directory: `C:\DCS-Dashboard`
- Arguments: *(leave blank)*

Details tab

- Display name: `DCS Web Dashboard`
- Startup type: `Automatic`

Log on tab

- Default **Local System** is usually fine. If you later get *Access Denied* reading DCS files, change it to the Windows account DCS runs under.

I/O tab *(optional but handy)*

- Output (stdout): `C:\DCS-Dashboard\logs\dashboard-out.log`
- Error (stderr): `C:\DCS-Dashboard\logs\dashboard-error.log`

Environment tab *(recommended — this is where your config lives)* Enter each variable on its own line, `KEY=value` :

```
JWT_SECRET=change-me-to-a-long-random-secret
APP_URL=http://YOUR_PUBLIC_IP:3001
DASHBOARD_ADDR=0.0.0.0:3001
GRPC_ENDPOINT=http://localhost:50051
ADMIN_PASSWORD=your_master_password
DCS_SAVED_GAMES_DIR=C:\Users\admin\Saved Games\DCS.openbeta_server
DCS_TASK_WHITELIST=DCS Web Server Console, DCSAdminBot, gRPC-LSO
DISCORD_CLIENT_ID=...
DISCORD_CLIENT_SECRET=...
DISCORD_GUILD_ID=...
DISCORD_ADMIN_ROLE_ID=role_id_1,role_id_2
```

3. Click **Install service**.

4. Open `services.msc`, find **DCS Web Dashboard**, right-click → **Start**.

To change config later: edit the service (`nssm edit "DCS Web Dashboard"`) and restart it.

8. Firewall & HTTPS

1. Open **Windows Defender Firewall with Advanced Security** → **Inbound Rules** → **New Rule**.

2. **Port** → **TCP** → **Specific local port**: `3001` → **Allow** → name it `DCS Web Dashboard (3001)` .

⚠ **HTTP vs HTTPS:** the dashboard serves plain **HTTP**. Over the public internet, login tokens and passwords would travel unencrypted. For public access, put a reverse proxy (nginx, Caddy, or Cloudflare Tunnel) in front to terminate **HTTPS**. In that setup:

- Bind the dashboard to `127.0.0.1:3001` (`DASHBOARD_ADDR=127.0.0.1:3001`) and **do not** open port 3001 to the internet — only the proxy reaches it.
- Set `APP_URL=https://your.domain` and use the matching `https://.../api/auth/callback` in Discord.

TLS termination and websockets are intentionally out of scope for the binary (the proxy handles TLS; the UI uses Server-Sent Events, not websockets).

9. First login

1. Browse to `http://YOUR_PUBLIC_IP:3001` (or your HTTPS domain).
2. Log in with the **master password** (`ADMIN_PASSWORD`) or **Sign in with Discord**.
3. On success you get a 7-day session and land on the **Server Status** page.

10. Admin guide — features

Authentication model

Two methods, both optional but you want at least one:

- **Master password** — set `ADMIN_PASSWORD` ; type it on the login page. Good as a fallback.
- **Discord OAuth** — members of your guild who hold an allowed role can sign in. Sessions last 7 days.

Every API call (except login, health, and the public telemetry streams) requires a valid bearer token, so the dashboard is closed to the public by default.

Server Status

Home page: server health, current active mission, uptime, and player count at a glance.

Mission Management

Control the server's mission queue:

- **Active mission** is read live from the DCS engine via gRPC and badged in the queue.
- **Queue** lists the `.miz` files from `serverSettings.lua` .
- **Run Now** loads any mission immediately, bypassing the queue.
- **Upload** pushes a local `.miz` from your PC to the server (stored under `Missions\Uploads`).

- **Browse Server** adds existing server-side `.miz` files to the queue.
- **Remove** drops a mission from rotation.

Weather (requires `DCS_DYNAMIC_WEATHER_DIR`)

GUI for the DCS-Dynamic-Weather Python script:

- **Current weather** cross-references the live mission with the script's output.
- **Presets** lists available profiles (temperature, QNH, wind, etc.).
- **Apply & Restart** injects a preset and cycles the mission, handling the **A/B mission swap** so Python can edit the inactive `.miz` without a Windows file-lock conflict.

Radar & Atmosphere (live gRPC streams)

- **Radar** — live plot of all units with distinct shapes (planes, helos, ships, ground) and highlighted human players. Streamed via SSE.
- **Atmosphere** — live wind/weather/atmospheric data from the running mission.

Events & Chat

- **Events** — live kill-feed, takeoffs, landings, crashes.
- **Chat** — read in-game multiplayer chat and send messages from the browser.

Players

Who's online, ping, coalition, and aircraft.

Tasks (Windows Task Scheduler remote control)

- Shows live status (`Ready` , `Running` , `Disabled`) of background tasks/bots.
- **Start / Stop / Restart** them from the browser.
- **Whitelist**: set `DCS_TASK_WHITELIST` so only named DCS tasks appear; anything else is hidden.
- **Guardrails**: actions require confirmation; tasks that would reboot the host are flagged loudly.

Server Settings

Visual editor for `serverSettings.lua` (name, password, description, max players, and advanced DCS flags). Changes are written to disk immediately, but **DCS only reloads them on the next mission cycle/restart**.

Access Logs

Security audit trail of every login attempt (written to `AUDIT_LOG_PATH`):

- **SUCCESS** — user logged in and got a session.
- **REJECTED** — user lacked the required role or wasn't in the guild. Capped at the most recent ~1,000 attempts.

11. Updating the dashboard

Updating the binary: stop the service, replace `rust-web-dashboard.exe`, start the service.

Re-embedding an updated frontend (only if the UI changed): the embedded UI is a snapshot of

`web-dashboard/out/` copied into the crate's `static/` folder.

1. In `web-dashboard`, build the static export (`next.config.ts` already sets `output: 'export'`):

```
npm install
npm run build
```

2. Replace the contents of `rust-web-dashboard\static\` with the new `web-dashboard\out\` tree.
3. Rebuild the binary (see §3 Option B) so the new assets are baked in.

After any dependency change, run `cargo audit` (and `cargo deny check` if configured) — the current build reports **0 vulnerabilities**.

12. Troubleshooting

Binary exits immediately with a `JWT_SECRET` error `JWT_SECRET` is unset or shorter than 16 characters. Set a long random secret and restart.

`could not find Cargo.toml when building` Your terminal opened at the repo root. Always pass `--manifest-path "C:\Users\thierry\Documents\GitHub\DCS-gRPC\rust-web-dashboard\Cargo.toml"`.

Repeating "failed to open StreamUnits/StreamEvents; retrying" / "tcp connect error" No DCS-gRPC server reachable at `GRPC_ENDPOINT`. Start the DCS-gRPC mod (or fix the endpoint). The UI still loads; live tabs populate once gRPC is up.

Discord login redirects to `localhost` `APP_URL` doesn't match your public address, or the Discord redirect URI is wrong. Set `APP_URL` to the real IP/host and add the exact `/api/auth/callback` URL in the Discord OAuth2 settings, then restart.

Settings changes don't show in-game `serverSettings.lua` is updated immediately, but DCS only re-reads it on mission cycle/restart.

"Access Denied" reading DCS files / applying weather Run the service under the Windows account that owns the DCS Saved Games folder (NSSM → *Log on* tab). For weather, ensure your queue's missions end in `_A.miz` / `_B.miz` so the A/B swap can edit the inactive file.

Tasks tab is empty or shows too much Check `DCS_TASK_WHITELIST` — when set, only matching task names appear; when unset, all root tasks are shown.